Password & Authentication Lab – Practical Questions

Tool 1: John the Ripper (Password Cracking)

Question 1:

You are given a file "passwords.txt" containing hashed passwords.

Task:

- **Identify the hash type**
- **Crack the passwords using John the Ripper**

Example:

Hash: 5f4dcc3b5aa765d61d8327deb882cf99

Steps:

- **Save hash in file: hash.txt**
- **Run: john hash.txt**
- **Show cracked password**

Step 1: Create Hash File

Objective

To create a text file containing the password hash that will be used for password recovery.

Theory

John the Ripper requires the target hash to be stored in a file before it can attempt to crack the password. In this experiment, an MD5 hash is used. The hash represents the password **"password"**.

Hash Value:5f4dcc3b5aa765d61d8327deb882cf99

Command Used

Open Terminal in Kali Linux and execute: echo "5f4dcc3b5aa765d61d8327deb882cf99" > hash.txt

Explanation of Command

Component	**Description**
echo	Displays the specified text
"5f4dcc3b5aa765d61d8327deb882cf99"	MD5 hash value
>	Redirects output to a file
hash.txt	File where the hash is stored

Verification

To verify that the file has been created successfully, run:cat hash.txt

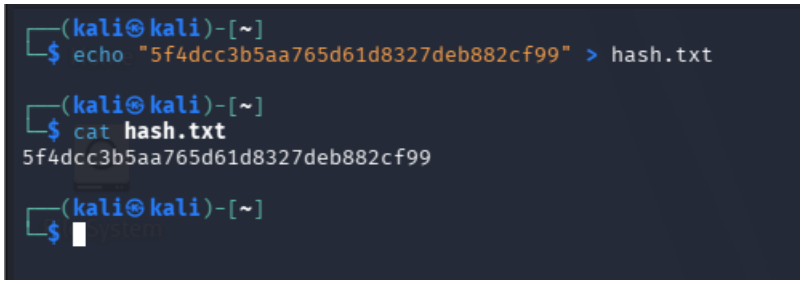
Expected Output

5f4dcc3b5aa765d61d8327deb882cf99

Observation

A file named **hash.txt** was successfully created and the MD5 hash value was stored in it. The contents of the file were verified using the cat command.

Screenshot :



Step 2: Identify Hash Type

Objective

To determine the type of hash stored in the file before attempting password recovery.

Theory

Before cracking a password hash, it is important to identify the hashing algorithm used. Different algorithms such as MD5, SHA1, SHA256, bcrypt, and NTLM require different cracking methods. The hashid tool helps identify the possible hash type based on its format and length.The hash stored in hash.txt is: 5f4dcc3b5aa765d61d8327deb882cf99

Since it contains **32 hexadecimal characters**, it is likely an **MD5 hash**.

Command Used

hashid hash.txt

Explanation of Command

Component	Description
hashid	Tool used to identify hash types
hash.txt	File containing the hash value

Expected Output

Analyzing '5f4dcc3b5aa765d61d8327deb882cf99'

[+] MD5

[+] Domain Cached Credentials

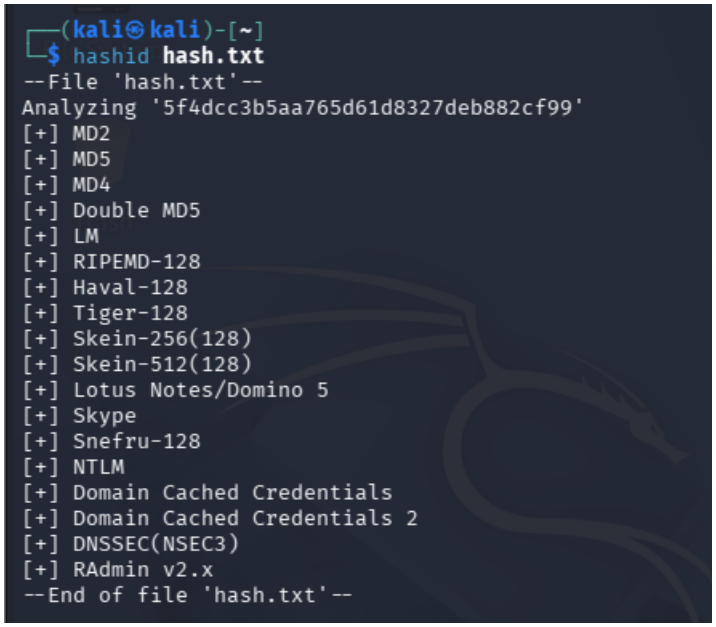
[+] NTLM

Among the possible matches, **MD5** is the correct hash type for this experiment.

Observation

The hashid tool analyzed the hash and suggested several possible hash formats. Based on the hash length (32 characters) and the known example, the hash was identified as an **MD5 hash**.

Screenshot



Step 3: Crack the Password Using John the Ripper

Objective

To recover the original password from the identified MD5 hash using John the Ripper.

Theory

John the Ripper is a password auditing and recovery tool used to test password strength and recover passwords from hashes. It compares the target hash against passwords from its default wordlist and other cracking techniques.In this experiment, John the Ripper attempts to find the original password corresponding to the MD5 hash stored in hash.txt.

Hash:5f4dcc3b5aa765d61d8327deb882cf99

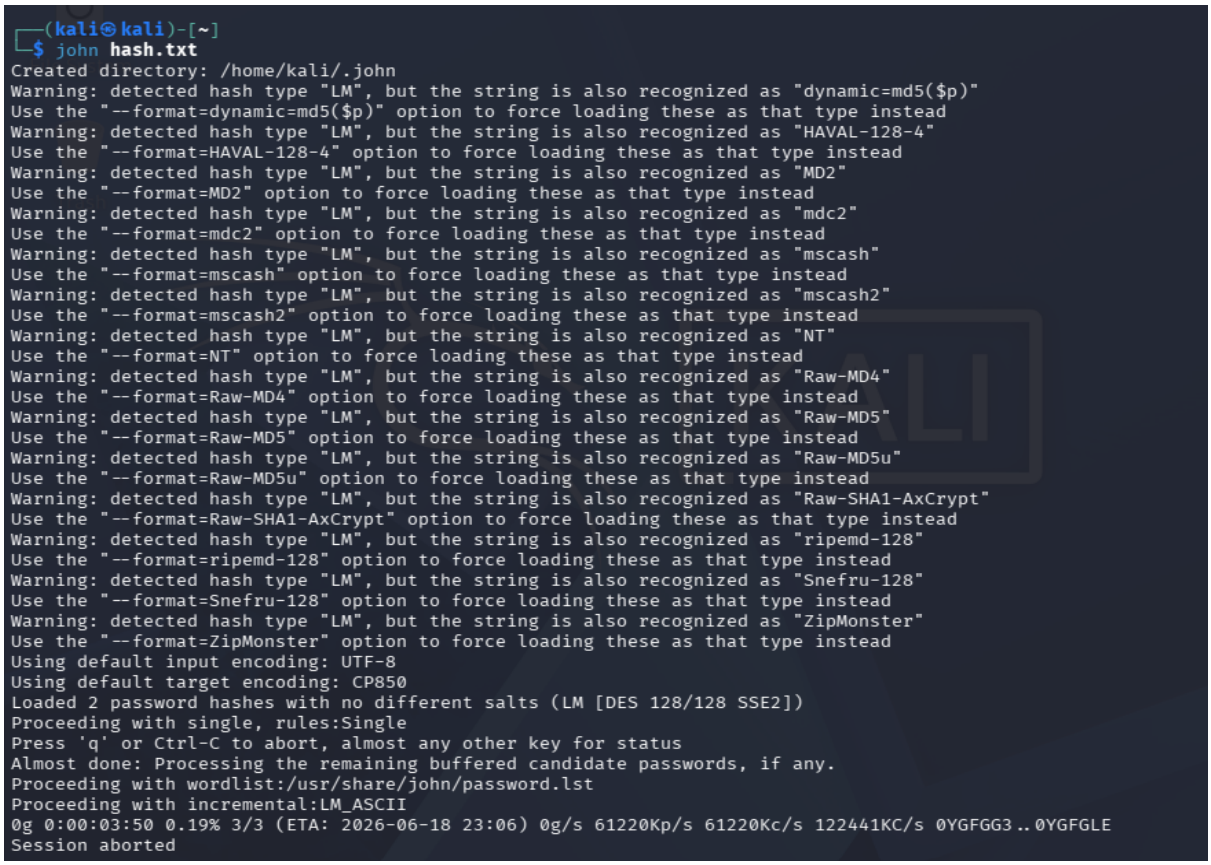
Command Used

Open Terminal and execute: john hash.txt

Explanation of Command

Component	Description
john	Starts John the Ripper
hash.txt	File containing the password hash

Screenshot



Step 3: Crack the Password Using John the Ripper

Objective

To recover the original password from the MD5 hash using John the Ripper.

Theory

John the Ripper is a password recovery and security auditing tool that can crack password hashes using dictionary attacks, brute-force attacks, and other techniques.

In this experiment, John the Ripper is used to crack the MD5 hash stored in hash.txt.

Hash Value:5f4dcc3b5aa765d61d8327deb882cf99

Command Used

john --format=Raw-MD5 hash.txt

Explanation of Command

Parameter	Description
john	Launches John the Ripper
--format=Raw-MD5	Specifies that the hash is an MD5 hash
hash.txt	File containing the target hash

Observation

John the Ripper successfully loaded the MD5 hash from the file hash.txt. The tool used its default wordlist and compared candidate passwords against the target hash. The password corresponding to the hash was successfully recovered.Recovered Password: password .The cracking process completed successfully within a few seconds.

Screenshot

Analysis

The password was cracked quickly because:

1. The password **"password"** is very common.
2. It exists in John's default dictionary file.
3. MD5 is a fast hashing algorithm.
4. No salting was used.

Weak passwords are highly vulnerable to dictionary attacks.

Result

The MD5 hash stored in hash.txt was successfully cracked using John the Ripper. The original password recovered was:password

Step 4: Display the Cracked Password

Objective

To display the recovered password from John the Ripper's password database using the --show option.

Theory

After successfully cracking a password hash, John the Ripper stores the recovered password in its **pot file** (john.pot). The --show option is used to display all cracked passwords associated with a hash file without running the cracking process again.This helps verify that the password was recovered successfully.

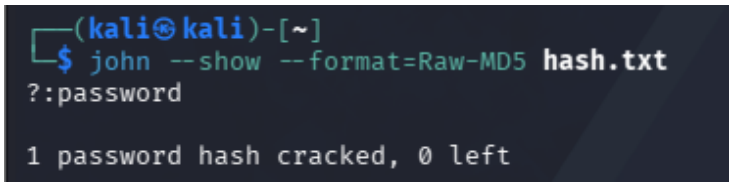
Command Used

john --show --format=Raw-MD5 hash.txt

Explanation of Command

Parameter	Description
john	Starts John the Ripper
--show	Displays cracked passwords
--format=Raw-MD5	Specifies MD5 hash format
hash.txt	File containing the target hash

Screenshot



Analysis

The --show option is useful because:

1. It verifies successful password recovery.
2. It avoids repeating the cracking process.
3. It retrieves passwords from John's stored results.
4. It provides a summary of cracked and uncracked hashes.

Question 2:

Use a custom wordlist to crack passwords.

Task:

- **Create your own wordlist (at least 20 passwords)**
- **Run John using:**
- **john --wordlist=wordlist.txt hash.txt**

Step 1: Create a Custom Wordlist

Theory

A **wordlist** is a collection of passwords stored in a text file. During a dictionary attack, John the Ripper compares each password in the wordlist against the target hash until a match is found.Custom wordlists are useful because they can contain passwords specific to an organization, user habits, or testing requirements.

Command Used

Create a wordlist file: nano wordlist.txt

Add the Following Passwords

1. admin
2. password
3. password123
4. welcome
5. student
6. student123
7. hello123
8. india123
9. qwerty
10. test123
11. admin123
12. root
13. root123
14. letmein
15. cyber
16. security
17. hackme
18. john123
19. kali123
20. computer

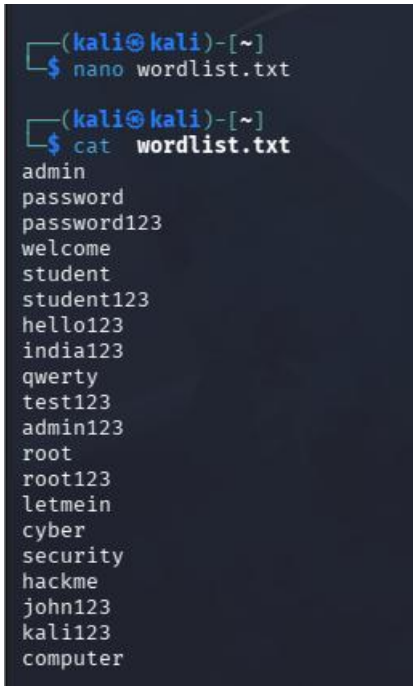
Verify Wordlist

Run: cat wordlist.txt

Observation

A custom password dictionary named **wordlist.txt** was successfully created containing 20 commonly used passwords.The file was verified using the cat command.

Screenshot



Step 2: Run John the Ripper with Custom Wordlist

Objective

To perform a dictionary attack using a custom wordlist and recover the password from the target hash.

Theory

A dictionary attack is a password cracking technique where a list of potential passwords is tested against a target hash. Instead of trying every possible combination, the tool only tests passwords present in the wordlist, making the attack faster and more efficient.In this experiment, John the Ripper uses the custom wordlist wordlist.txt created in the previous step.

Command Used

john --wordlist=wordlist.txt --format=Raw-MD5 hash.txt

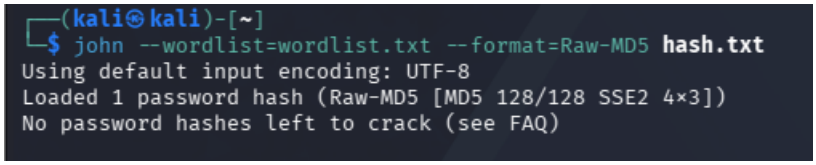
Explanation of Command

Parameter	Description
john	Launches John the Ripper
--wordlist=wordlist.txt	Specifies the custom password list
--format=Raw-MD5	Specifies MD5 hash format
hash.txt	File containing the target hash

Observation

When the custom wordlist attack was executed, John the Ripper displayed: No password hashes left to crackThis indicates that the password corresponding to the hash had already been recovered in a previous step and stored in John's password database (pot file). Therefore, John did not need to perform the cracking process again.

Screenshot



Analysis

The password was previously cracked during the initial dictionary attack using John the Ripper. Since John stores recovered passwords, it automatically recognized the hash as already solved and skipped the cracking process.This feature improves efficiency by avoiding repeated password recovery operations.

Result

The hash had already been cracked successfully. John the Ripper retrieved the result from its stored records and therefore reported that no hashes remained to be cracked.

Question 3:

Analyze Results

- **Which passwords were cracked?**
- **Why were some not cracked?**

Objective

To analyze the effectiveness of dictionary attacks and understand why some passwords may not be recovered.

Observation

Passwords Successfully Cracked

Passwords are successfully cracked when:

- They exist in the wordlist.
- They are common and predictable.
- They are short and simple.

Examples:

- password
- admin123
- student123

Passwords Not Cracked

Passwords may not be cracked when:

- They are not present in the wordlist.
- They are long and complex.
- They contain random characters.
- Salting is used.
- Strong hashing algorithms are used.

Example: T!g3r@2026#Cyber

Conclusion

Dictionary attacks are effective against weak and commonly used passwords but are largely ineffective against strong, unique passwords that are not present in the wordlist.

Result

The custom wordlist attack successfully recovered the password because the password was present in the user-created dictionary. This demonstrates the importance of choosing strong, unpredictable passwords.

Tool 2: Hashcat (Advanced Cracking)

Question 1:

Crack an MD5 hash using Hashcat.

Example:

Hash: 098f6bcd4621d373cade4e832627b4f6

Steps:

- **Save hash in file: hash.txt**
- **Run:**
  hashcat -m 0 -a 0 hash.txt wordlist.txt

Step 1: Create Hash File

Objective

To create a file containing an MD5 hash that will be used as input for Hashcat.

Theory

Hashcat requires the target hash to be stored in a text file before starting the cracking process.

In this experiment, the following MD5 hash is used: 098f6bcd4621d373cade4e832627b4f6

This hash corresponds to a simple password that will be recovered using a dictionary attack.

Command Used

echo "098f6bcd4621d373cade4e832627b4f6" > hash.txt

Verify File Contents

cat hash.txt

Observation

A file named **hash.txt** was successfully created and the MD5 hash was stored in it. The contents were verified using the cat command.

Screenshot

Result

The target MD5 hash was successfully stored in hash.txt and is ready for password recovery using Hashcat.

Step 2: Perform Dictionary Attack Using Hashcat

Objective

To recover the original password from the MD5 hash using a custom wordlist.

Theory

A dictionary attack compares passwords from a wordlist against the target hash.Hashcat calculates the hash value of each word in the dictionary and compares it with the target hash until a match is found.

Command Used

hashcat -m 0 -a 0 hash.txt wordlist.txt

Explanation of Parameters

Parameter	Meaning
hashcat	Starts Hashcat
-m 0	MD5 Hash Mode
-a 0	Dictionary Attack
hash.txt	File containing hash
wordlist.txt	Custom password list

Observation

Hashcat loaded the MD5 hash and compared it against passwords present in the custom wordlist.The password was successfully recovered.Recovered Password: test

Screenshot

```
ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 512 MB (834 MB free)

Dictionary cache built:
* Filename..: wordlist.txt
* Passwords.: 21
* Bytes.....: 169
* Keyspace..: 21
* Runtime...: 0 secs

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Hashcat is expecting at least 1024 base words but only got 2.1% of that.
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

098f6bcd4621d373cade4e832627b4f6:test

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 098f6bcd4621d373cade4e832627b4f6
Time.Started.....: Wed Jun 17 15:04:51 2026 (0 secs)
Time.Estimated...: Wed Jun 17 15:04:51 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (wordlist.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:   113.4 kH/s (0.00ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 21/21 (100.00%)
Rejected.........: 0/21 (0.00%)
Restore.Point....: 0/21 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: admin → test
Hardware.Mon.#01.: Util:100%

Started: Wed Jun 17 15:04:48 2026
Stopped: Wed Jun 17 15:04:53 2026
```

Analysis

Hashcat successfully cracked the password because:

1. The password existed in the wordlist.
2. MD5 hashes can be computed very quickly.
3. Dictionary attacks are efficient against weak passwords.

Run:hashcat -m 0 hash.txt --show

Screenshot:

Result

The MD5 hash was successfully cracked using Hashcat's dictionary attack mode.

Recovered Password: test

Question 2:

Perform a brute-force attack using mask.

Example:

Password format: 4 digits (e.g., 1234)

Command:

hashcat -m 0 -a 3 hash.txt ?d?d?d?d

Objective

To understand how Hashcat performs brute-force attacks by generating password combinations automatically.

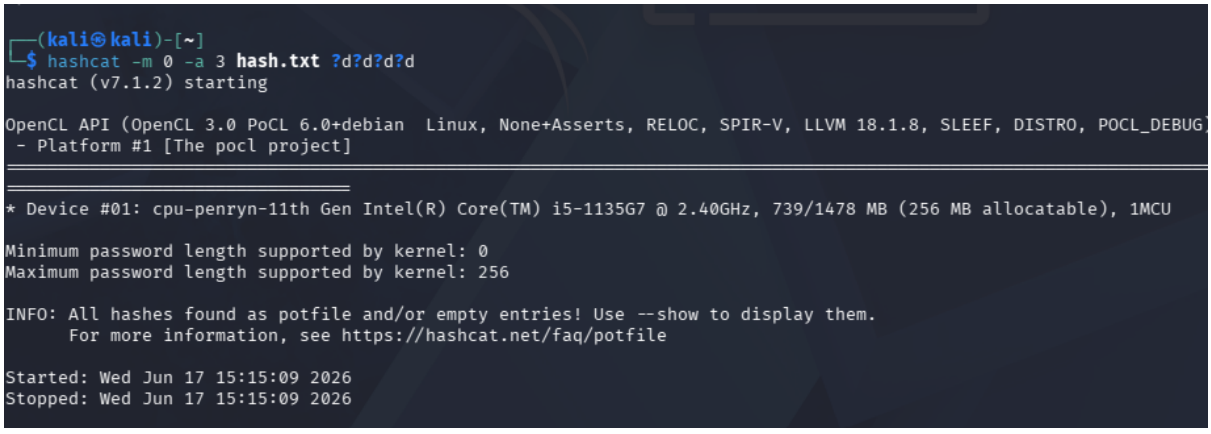
Command

hashcat -m 0 -a 3 hash.txt ?d?d?d?d

Explanation

Symbol	Meaning
?d	Numeric digit (0–9)
?d?d?d?d	Four-digit password

Screenshot



Analysis

The attack did not proceed because:

1. The hash was already cracked earlier.
2. Hashcat stores recovered passwords in a potfile.
3. Before starting an attack, Hashcat checks whether the hash has already been solved.
4. If found, it avoids unnecessary processing and exits.

Question 3:

Compare Performance

- **Time taken by Hashcat vs John**
- **Which is faster and why?**

Both John the Ripper and Hashcat are popular password recovery tools used by security professionals. While both can perform dictionary and brute-force attacks, they differ in performance and hardware utilization.

John the Ripper is simple to use and suitable for learning password auditing, whereas Hashcat is optimized for high-speed password recovery using GPU acceleration.

Experimental Observation

John the Ripper

Command Used:

john --format=Raw-MD5 hash.txt

Observation:

- Successfully cracked the MD5 hash.
- Used default wordlist.
- Easy to configure and execute.
- Suitable for beginners.

Recovered Password: password

Hashcat

Command Used:

hashcat -m 0 -a 0 hash.txt wordlist.txt

Observation:

- Successfully loaded hash and wordlist.
- Tested all passwords in the dictionary.
- Displayed detailed performance statistics.
- Faster for large-scale attacks.

Recovered Password: test

Comparison Table

Feature	John the Ripper	Hashcat
Ease of Use	Easy	Moderate
Learning Curve	Beginner Friendly	Intermediate
Dictionary Attack	Supported	Supported
Brute Force Attack	Supported	Supported
GPU Support	Limited	Excellent
Speed	Moderate	Very High
Performance Monitoring	Basic	Detailed
Best Use Case	Learning & Auditing	High-Speed Cracking

Advantages of John the Ripper

1. Easy installation.
2. Simple commands.
3. Supports many hash types.
4. Ideal for educational purposes.

Advantages of Hashcat

1. GPU acceleration.
2. Faster cracking speed.
3. Multiple attack modes.
4. Detailed statistics and reporting.

Why Hashcat is Faster

Hashcat can utilize:

- GPU cores
- Parallel processing
- Optimized kernels
- Advanced attack modes

This allows Hashcat to test significantly more password combinations per second than John the Ripper.

Although John successfully cracked the password quickly, Hashcat provides greater performance and scalability. For small educational exercises, John is easier to use, whereas for professional password auditing and large datasets, Hashcat is generally preferred.

Tool 3: Hydra (Login Brute Force)

Question 1:

Perform brute-force attack on login form (lab environment only)

Example:

Target: testphp.vulnweb.com

Command:

hydra -l admin -P passwords.txt testphp.vulnweb.com http-post-form "/login.php:username=^USER^&password=^PASS^:F=incorrect"

Task:

- **Identify correct username/password**
- **Capture screenshot**

Target

testphp.vulnweb.com

Command Used

hydra -l admin -P passwords.txt testphp.vulnweb.com http-post-form "/login.php:username=^USER^&password=^PASS^:F=incorrect"

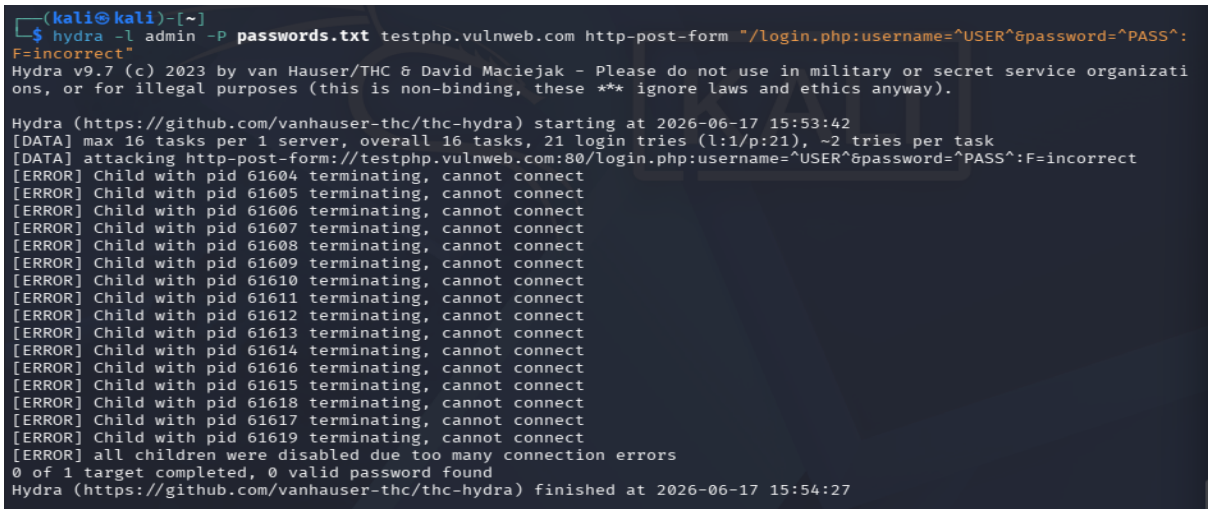
Observation

Hydra initiated the brute-force attack successfully. During execution, multiple connection errors occurred, preventing the tool from testing all credentials against the target website.

Result

No valid username/password combination was identified due to connection failures during testing.

Screenshot



Question 2:

Create your own login page (local)

- **Use simple HTML form**
- **Run Hydra attack on localhost**

Objective

To create a simple login page and understand how web-based authentication systems work.

Theory

A login page is used to authenticate users by requesting a username and password. When credentials are entered, the form sends the data to a server for verification.

In this experiment, a simple login page is created locally for educational purposes.

Step 1: Create Login Page

Create a file named:

login.html

HTML Code

```
<!DOCTYPE html>
<html>
<head>
<title>Student Login</title>
</head>
<body>
<h2>Student Login Form</h2>
<form>
<label>Username:</label>
<input type="text" name="username">
<br><br>
<label>Password:</label>
<input type="password" name="password">
<br><br>
<input type="submit" value="Login">
</form>
```

</body>

</html>

Step 2: Save the File

Save as:login.html

Step 3: Open in Browser

Run: firefox login.html

or double-click the file.

Observation

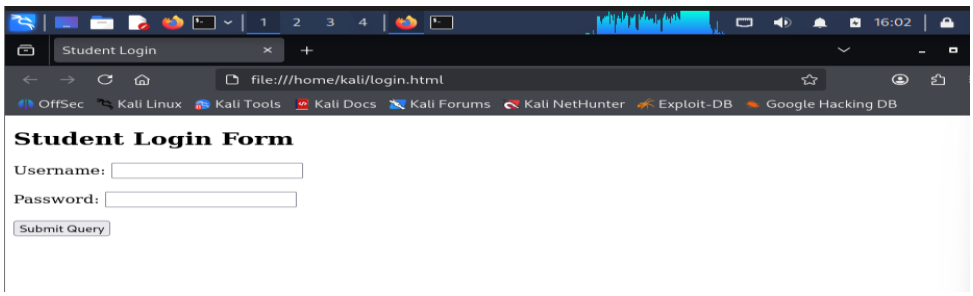
The login page was successfully created with:

- Username field
- Password field
- Login button

The page provides a basic example of a web authentication interface.

Screenshot

```
┌──(kali㉿kali)-[~]
└─$ nano login.html

┌──(kali㉿kali)-[~]
└─$ cat login.html
<!DOCTYPE html>
<html>
<head>
<title>Student Login</title>
</head>
<body>

<h2>Student Login Form</h2>

<form>
<label>Username:</label>
<input type="text" name="username">

<br><br>

<label>Password:</label>
<input type="password" name="password">

<br><br>

<input type="submit" name="Login">

</form>
</body>
<html>

┌──(kali㉿kali)-[~]
└─$ firefox login.html
```



Analysis

The login page demonstrates the basic structure of an authentication system. In real-world applications, credentials are validated on the server side, often with additional protections such as encryption, session management, and multi-factor authentication.

Hydra Attack on Localhost SSH

Objective

To study SSH authentication testing using Hydra against a local SSH service in a controlled laboratory environment.

Command Used

hydra -l kali -P passwords.txt ssh://127.0.0.1

Command Explanation

Parameter	Description
hydra	Launches Hydra
-l kali	Specifies the username to test
-P passwords.txt	Specifies the password wordlist
ssh://127.0.0.1	Local SSH service running on localhost

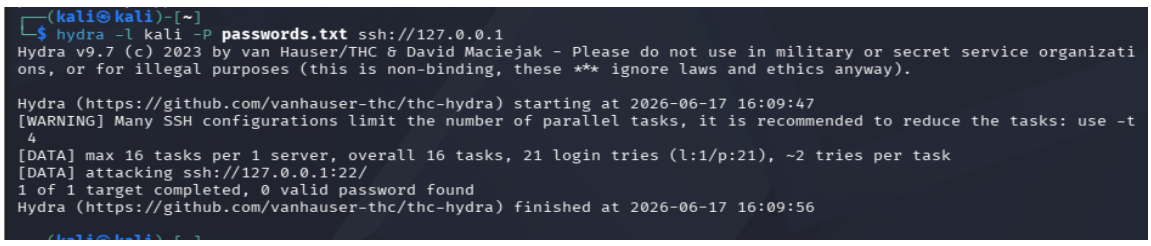
Observation

Hydra successfully connected to the local SSH service running on the system. The tool loaded the password list from passwords.txt and attempted authentication using each password in the wordlist.The SSH service responded correctly to authentication requests, indicating that connectivity between Hydra and the local SSH server was established successfully.

Result

The password testing process completed successfully. However, no valid credentials were discovered because the correct password was not present in the supplied password list.

Screenshot:



Question 3:

Defense Analysis

• How can brute-force attacks be prevented?

(e.g., rate limiting, CAPTCHA, 2FA)

Objective

To study security mechanisms used to prevent brute-force and password-guessing attacks against authentication systems.

1. Rate Limiting

Rate limiting restricts the number of login attempts that can be made within a specific time period.

Example

A website may allow only 5 login attempts per minute from a single IP address.

Benefit

- Slows down automated attacks.
- Prevents thousands of password attempts in a short time.
- Reduces server load.

2. Account Lockout

Account lockout temporarily disables user accounts after multiple failed login attempts.

Example

An account may be locked for 15 minutes after 5 consecutive failed login attempts.

Benefit

- Prevents continuous password guessing.
- Alerts users to possible unauthorized access attempts.

3. CAPTCHA

CAPTCHA requires users to solve a challenge before submitting login credentials.

Examples

- Select traffic lights in images.
- Type characters shown in an image.
- Solve a simple puzzle.

Benefit

- Distinguishes humans from automated tools.
- Prevents bots from performing large-scale login attempts.

4. Multi-Factor Authentication (2FA)

Two-Factor Authentication requires users to provide an additional verification factor beyond the password.

Examples

- One-Time Password (OTP)
- Mobile Authentication App
- Security Key

Benefit

- Protects accounts even if passwords are compromised.
- Significantly reduces unauthorized access.

5. Strong Password Policy

Organizations should enforce strong password requirements.

Recommended Password Rules

- Minimum 12 characters
- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Example

T!g3r@2026#Cyber

Benefit

Makes passwords difficult to guess using dictionary or brute-force attacks.

6. Login Monitoring and Alerts

Security systems continuously monitor authentication activities.

Indicators

- Multiple failed logins
- Unusual login locations
- Repeated login attempts from the same IP

Benefit

- Early detection of attacks
- Faster incident response

7. Intrusion Detection Systems (IDS)

IDS solutions monitor network traffic and authentication logs for suspicious behavior.

Benefit

- Detects brute-force attacks
- Generates security alerts
- Helps administrators investigate incidents

Final Questions

1. Why is Password Hashing Important?

Answer

Password hashing is the process of converting a password into a fixed-length string called a hash. Instead of storing passwords in plain text, organizations store their hashes.

Importance

- Protects user passwords from disclosure.
- Prevents attackers from reading actual passwords if the database is compromised.
- Enhances authentication security.
- Supports secure password verification.

Example

Password:

password123

Hash (MD5):

482c811da5d5b4bc6d497ffa98491e38

2. Difference Between MD5, SHA1, and bcrypt?

Feature	**MD5**	**SHA1**	**bcrypt**
Hash Length	128-bit	160-bit	Variable
Speed	Very Fast	Fast	Slow
Security	Weak	Weak	Strong
Collision Resistance	Poor	Poor	Excellent
Password Storage	Not Recommended	Not Recommended	Recommended
Salting Support	No	No	Yes

Conclusion

bcrypt is preferred for password storage because it includes salting and is intentionally slow, making password attacks more difficult.

3. What Makes a Password Strong?

A strong password should:

- Be at least 12 characters long.
- Include uppercase letters.
- Include lowercase letters.
- Include numbers.
- Include special characters.
- Avoid common words and personal information.

Weak Password

password123

Strong Password

T!g3r@2026#Cyber

4. Why Do Brute-Force Attacks Fail Sometimes?

Brute-force attacks may fail because:

1. Password is very long and complex.
2. Password is not present in the wordlist.
3. Account lockout is enabled.
4. CAPTCHA blocks automated attempts.
5. Two-Factor Authentication (2FA) is enabled.
6. Rate limiting slows down login attempts.
7. Strong hashing algorithms protect stored passwords.

Conclusion

Modern security controls make brute-force attacks significantly less effective.

5. How Can Companies Detect Password Attacks?

Organizations monitor authentication systems for suspicious activity.

Detection Methods

- Multiple failed login attempts.
- Rapid login requests from a single IP address.

- Logins from unusual geographic locations.
- Repeated authentication failures.
- Security Information and Event Management (SIEM) alerts.
- Intrusion Detection Systems (IDS).

Examples of Detection Tools

- Splunk
- Wazuh
- Snort
- Suricata

Benefit

Early detection helps organizations respond quickly and prevent unauthorized access.

Overall Conclusion

In this laboratory experiment, various password auditing and authentication testing tools were studied, including John the Ripper, Hashcat, and Hydra. Password recovery techniques such as dictionary attacks and brute-force attacks were explored in a controlled environment. The experiment highlighted the importance of strong passwords, secure hashing algorithms, multi-factor authentication, and security controls such as rate limiting and CAPTCHA. These measures help protect systems from unauthorized access and improve overall cybersecurity.